

```

graph LR
    subgraph LoginForm [Please sign in]
        direction TB
        Username[Username: Basant]
        Password[Password: Pwd1]
        SignIn[Sign in]
    end
    Application[Application  
file: /products/ibid]
    URL[mybatis.org/dist/ibatis-2.3.0-RC2/ibatis-2.3.0-RC2.jar]

    SignIn --> Application
    Application --> URL
  
```

The image shows a JWT token and its decoded components. On the left, the token is displayed in a text area: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0MTI0dXMtZGF1ZS1kIiwiaWF0IjoxNTE2MzQ1MDIyMC5kaWJJSMeKfZQT47qWpJ6F3Pk6yJV_adQsw5c`. On the right, a table displays the decoded header and payload.

	HEADER: ALGORITHM & TOKEN TYPE
	<pre>{ "alg": "HS256", "typ": "JWT" }</pre>
	PAYLOAD: DATA
	<pre>{ "sub": "123456789", "name": "John Doe", "iat": 1516239022 }</pre>
	VERIFY SIGNATURE
	<pre>WRACDH256{ base64url_encode(header) + "." + base64url_encode(payload), your-256-bit-secret } ☐ secret base64 encoded</pre>

GUID

MachineKey

WPA Key

WEP Key

Encryption key

Password

Security level

64-bit

128-bit

256-bit

512-bit

1024-bit

2048-bit

4096-bit

Hex ?

How many ?

☒ Yes

1

Result

■

5367566B59703373367639792F423F4528482B4D6251655468576D5A71347487

<https://github.com/Java-Techie-jt/spring-boot3-jwt/blob/main/src/main/java/com/javatechie/service/JwtService.java>

This is user created class – which contain generate Token method and utility methods around it. Generate token using username, expiry date and **signatureAlgorithm** and **SECRET**. These 3 jars we require for this purpose

```
56
57 public String generateToken(String userName){
58     Map<String,Object> claims=new HashMap<>();
59     return createToken(claims,userName);
60 }
61
62 private String createToken(Map<String, Object> claims, String userName) {
63     return Jwts.builder()
64         .setClaims(claims)
65         .setSubject(userName)
66         .setIssuedAt(new Date(System.currentTimeMillis()))
67         .setExpiration(new Date(System.currentTimeMillis()+1000*60*30))
68         .signWith(getSignKey(), SignatureAlgorithm.HS256).compact();
69 }
70
71 private Key getSignKey() {
72     byte[] keyBytes= Decoders.BASE64.decode(SECRET);
73     return Keys.hmacShaKeyFor(keyBytes);
74 }
```

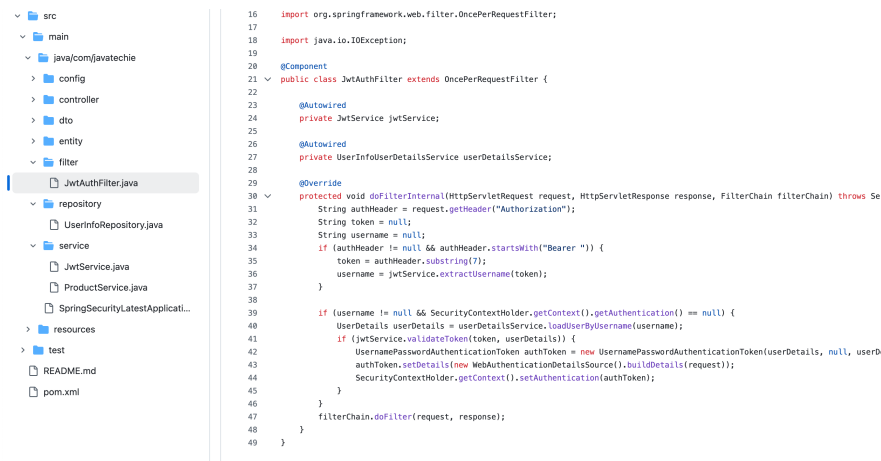
```
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.11.5</version>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId>
<version>0.11.5</version>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId>
<version>0.11.5</version>
</dependency>
```

Build validate token method which verifies token has valid user and is not yet expiry.

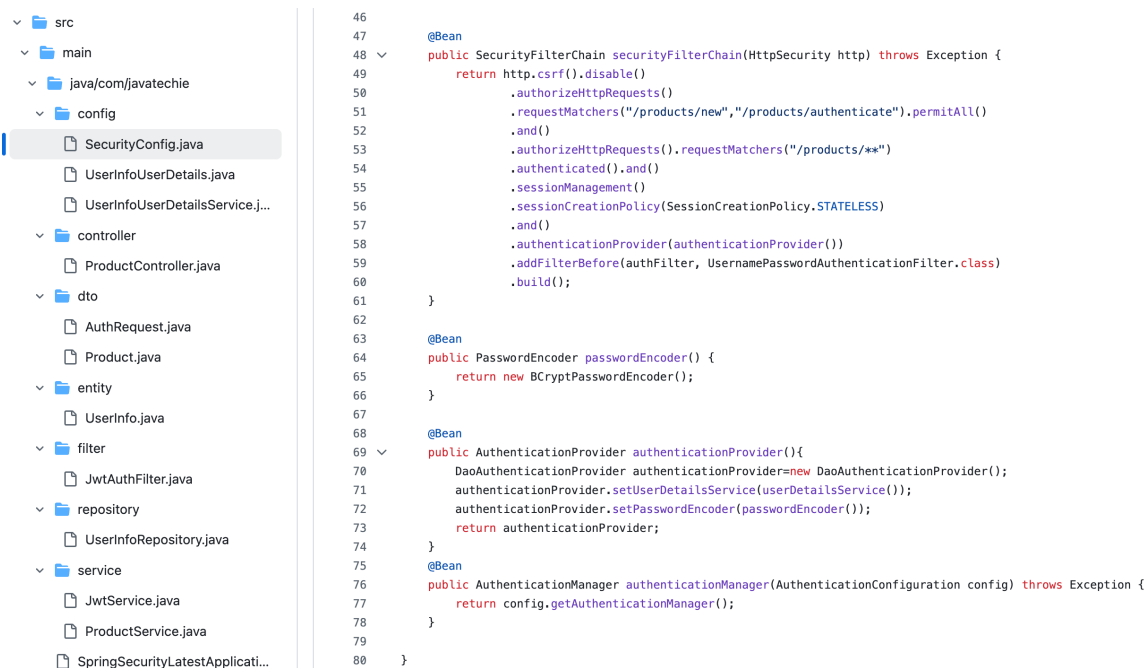
```
18 @Component
19 public class JwtService {
20
21
22     public static final String SECRET = "5367566B59703373367639792F423F4528482B4D6251655468576D5A71347437";
23
24
25     public String extractUsername(String token) {
26         return extractClaim(token, Claims::getSubject);
27     }
28
29     public Date extractExpiration(String token) {
30         return extractClaim(token, Claims::getExpiration);
31     }
32
33     public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
34         final Claims claims = extractAllClaims(token);
35         return claimsResolver.apply(claims);
36     }
37
38     private Claims extractAllClaims(String token) {
39         return Jwts
40             .parserBuilder()
41             .setSigningKey(getSignKey())
42             .build()
43             .parseClaimsJws(token)
44             .getBody();
45     }
46
47     private Boolean isTokenExpired(String token) {
48         return extractExpiration(token).before(new Date());
49     }
50
51     public Boolean validateToken(String token, UserDetails userDetails) {
52         final String username = extractUsername(token);
53         return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));
54     }
55 }
```

Intercept user request and validate the token – using JwtAuthFilter

Extract user from token – check authentication object in SpringContextHolder if not load user from DB using UserDetailService



Use this JwtAuthFilter in the SecurityFilter Bean



Username and password authentication request

